

MULTI-SYMBOL

Parallel Monitoring & Concurrent Trade Execution

*How the Order Flow Engine handles 100s of stocks simultaneously,
live tick data, independent symbol processing, and automated concurrent trading*

v1.0 | May 2026 | Architecture & Engineering Reference

1. The Challenge — Why This Is Not Trivial

1.1 Scale of the Problem

When a user asks to monitor 100 stocks simultaneously with live order flow indicators and automated trading, the engine faces a qualitatively different challenge than single-instrument operation. The numbers make this clear:

Metric	Single Symbol	100 Symbols	500 Symbols (max tier)
Live tick streams	1	100	500
Ticks per second (market open)	~50–500	~5,000–50,000	~25,000–250,000
Ticks per session (6.5hr RTH)	~50,000	~5,000,000	~25,000,000
Computations per tick (7+ ops)	7	700	3,500
Simultaneous bar types per symbol	1–4	100–400	500–2,000
Active OFE-Scripts to evaluate	1–5	100–500	500–2,500
Volume Profile updates per minute	12 (5-min bars)	1,200	6,000
VWAP recalculations per second	0.2 (5-sec interval)	20	100
Potential trade signals per hour	1–10	100–1,000	500–5,000
Concurrent trade executions possible	1	1–20 simultaneous	1–100 simultaneous

A naive single-threaded engine cannot process 50,000 ticks/second. Each tick would queue behind the previous one, introducing latency measured in SECONDS by market open. By the time a signal fires, the price has moved. This is architecturally unacceptable for trading.

1.2 The Core Architectural Requirement

To address monitoring 100s of stocks simultaneously with live tick data and concurrent automated trading, the engine must be redesigned as a Symbol-Sharded Parallel Processing System with these guarantees:

Guarantee	What It Means	Why Critical for Trading
Symbol Isolation	Each symbol's entire state (bars, delta, profile, signals) is processed on its own	A crash, stall, or slow feed on AAPL cannot delay signal detection on TSLA. Each symbol is an independent universe.

	independent thread/core with no shared mutable state	
Zero Tick Loss	No tick from any symbol is ever dropped, even at peak load. Back-pressure handling prevents overflow.	A missed tick corrupts delta, CVD, and imbalances for the rest of the session. One dropped tick = wrong signals all day.
Consistent Latency	Signal detection latency is < 10ms from tick receipt to signal emission, at any number of active symbols	Trading automation acts on signals. A 500ms delayed signal on a fast-moving stock means entry at a worse price or a missed trade.
True Concurrency	Tick processing for Symbol A and Symbol B runs truly in parallel on separate CPU cores — not interleaved on a single core	Interleaving introduces jitter. Under load, a bursty symbol (e.g. post-earnings news) would starve quiet symbols.
Independent Script Execution	Each user's automated script for each symbol runs in its own evaluation context. Scripts do not block each other.	A script monitoring 100 symbols fires when ANY of them signals. That firing must not delay evaluation of the other 99.
Cross-Symbol Correlation (optional)	The engine can optionally surface correlations across symbols (e.g. sector-wide delta shift) without coupling their individual processing	Advanced strategies monitor sector rotation. AAPL and MSFT moving together with opposite delta = institutional rotation signal.

2. Engine Architecture — Symbol-Sharded Parallel Design

2.1 Architecture Overview

The engine uses a Symbol Shard model: each active symbol is assigned to an independent processing unit called a Symbol Worker. Workers run in parallel, share no mutable state, and communicate only through message queues.

Layer	Component	Technology	Role
Ingestion	Tick Router	C++ — lock-free MPMC queue per symbol	Receives raw ticks from data provider, routes each tick to the correct Symbol Worker's queue by instrument ID
Processing	Symbol Worker Pool	C++ — one thread per symbol (or one per CPU core with symbol multiplexing)	Each Symbol Worker owns the complete state for one symbol: bars, delta, CVD, Volume Profile, VWAP, all signals
Detection	Signal Detector (per worker)	C++ — runs inside Symbol Worker thread	Evaluates all configured signals for the symbol after every bar close or tick (for tick-level signals)
Script Engine	OFE-Script Evaluator	Rust — isolated per-script process	Evaluates user's composite conditions against signal outputs. Each script gets its own evaluator instance.
Distribution	Event Bus	Redis Streams or Apache Kafka	Symbol Workers publish signals to the event bus. API servers subscribe and fan out to WebSocket clients.
API Layer	WebSocket Servers	uWebSockets C++ or Bun.js	Push signals to subscribing clients. Stateless — can be scaled horizontally.
Execution	Trade Router	C++ — priority queue per symbol	Receives trade instructions from user scripts, validates against risk limits, routes to broker adapter
Persistence	Time-Series DB	TimescaleDB + Redis	All ticks and signals persisted. Redis caches latest state per symbol for fast API reads.

2.2 Symbol Worker — Deep Dive

The Symbol Worker is the atomic unit of the engine. Each active symbol has exactly one Symbol Worker. Understanding its internal structure is essential to understanding the scaling model.

Symbol Worker Internal State (per symbol)

State Component	Description	Memory (typical)	Reset Behavior
Tick Ring Buffer	Lock-free SPSC ring buffer holding last 10,000 ticks. The Tick Router writes; the Worker reads.	~320 KB (10,000 × 32 bytes)	Never — circular overwrite
Active Bars (per bar type/size)	Up to 4 simultaneous bar types open. Each bar accumulates price levels as ticks arrive.	~50 KB per bar type	Resets on bar close
Delta State	Running bar_delta, max_delta, min_delta, cumulative_delta for session	< 1 KB	bar_delta resets on bar close; CVD resets on session open
Volume Profile (all types)	Bid/ask volume per price level for session, daily, weekly, flexible profiles	~200 KB per profile type (500 price levels × bid/ask)	Session profile resets at session_open
VWAP Engine	All active VWAP calculations: daily, weekly, yearly, anchored variants	~10 KB	Daily VWAP resets at session_open
Signal State	Last N signal occurrences per signal type; zone IDs for persistent zones; magnet active flags	~100 KB	Some persist across sessions (zones, magnets); others reset daily
Script Bindings	References to OFE-Scripts active for this symbol. Script evaluation triggered after each signal.	~1 KB per script	Updated when user adds/removes scripts
Imbalance Zone Registry	Active stacked imbalance zones with zone_id, price range, creation time, strength score	~50 KB (hundreds of zones)	Zones persist until price resolves them or session ends

i Total memory per Symbol Worker: approximately 1–2 MB including all state. For 500 symbols: 500–1,000 MB (0.5–1 GB) of working memory. This fits comfortably on a modern server with 64–128 GB RAM.

2.3 Tick Processing Pipeline — Per Symbol Worker

Every tick goes through this exact pipeline inside the Symbol Worker. Total pipeline latency target: < 2ms end-to-end per tick.

Stage	Operation	Latency Budget	Notes
1. Tick Receipt	Tick arrives in Symbol Worker's ring buffer from Tick Router	< 50 μ s	Lock-free write by router; lock-free read by worker
2. Ready Classification	Classify tick as BID-side or ASK-side aggression	< 5 μ s	Comparisons against prevailing bid/ask from exchange quote
3. Bar Accumulation	Update price level record for current bar(s). Increment bid_vol or ask_vol at this price level.	< 10 μ s	Up to 4 bar types simultaneously — all update on every tick
4. Delta Update	Recalculate bar_delta, max_delta, min_delta, cumulative_delta	< 1 μ s	Simple arithmetic on running counters — near-zero cost
5. VWAP Update	Update all active VWAP calculations with new price $\times$ volume	< 5 μ s	Incremental update formula — no full recalculation
6. Volume Profile Update	Add tick volume to the correct price bucket in active profiles	< 5 μ s	Hash lookup by price $\rightarrow O(1)$
7. Tick-Level Signal	Evaluate tick-level signals: VWAP Reaction touch, Turns detection, Delta Surge threshold	< 20 μ s	Only for signals that fire intra-bar, not on bar close

Check			
8. Bar Close Check	Is this tick the last tick before a TIME/RANGE/VOLUME boundary?	< 1 μ s	Simple comparison
9. Bar Close Processing (if applicable)	Finalize bar. Run all bar-close signal detectors: Pulse, imbalances, stacked check, absorption, COT, single print, ratio.	< 500 μ s	Only on bar close — most expensive stage. All signal detectors run here.
10. Signal Emission	Publish detected signals to Event Bus	< 100 μ s	Serialise to Protobuf/FlatBuffers, write to Redis Stream or Kafka topic
11. Script Evaluation Trigger	Notify OFE-Script evaluator that new signals are available for this symbol	< 50 μ s	Non-blocking message to script evaluator subprocess

***i** Stages 1–8 run on every tick. Stage 9–11 run only on bar close. At 500 ticks/second on one symbol, stages 1–8 consume < 1ms of CPU time per second — leaving 99%+ of the core available for bar-close processing and other symbols.*

2.4 Horizontal Scaling Model

The Symbol Worker Pool scales in two dimensions:

Vertical Scaling — Threads per Server

Server Config	Symbol Workers	Symbols Supported	Tick Capacity/sec
4-core / 16GB RAM	4 workers (1 per core)	Up to 50 symbols (time-sharing)	~25,000 ticks/sec

8-core / 32GB RAM	8 workers	Up to 100 symbols	~50,000 ticks/sec
16-core / 64GB RAM	16 workers	Up to 200 symbols	~100,000 ticks/sec
32-core / 128GB RAM	32 workers	Up to 500 symbols	~250,000 ticks/sec
64-core / 256GB RAM	64 workers	Up to 1,000 symbols	~500,000 ticks/sec

Horizontal Scaling — Multiple Engine Servers

Deployment	Servers	Total Symbols	Failover
Starter (v1.0 NinjaTrader launch)	1 × 8-core server	100 symbols	Manual restart — acceptable for v1.0
Growth (post-launch)	2 × 16-core servers	400 symbols	Active-passive failover
Scale (Platform expansion)	4 × 32-core servers	2,000 symbols	Active-active with consistent hashing
Enterprise	N × 32-core servers	N × 500 symbols (unlimited)	Full redundancy, zero-downtime deploys

i Symbol assignment to servers uses consistent hashing on the symbol ID. When a server is added, approximately 1/N of symbols migrate — minimising disruption. Each symbol's state is checkpointed to TimescaleDB every 60 seconds, so failover loses at most 60 seconds of in-memory state (which is reconstructed from tick replay).

3. User-Facing API — Monitoring Multiple Symbols Simultaneously

3.1 Symbol Watchlist — Register Multiple Symbols at Once

A user registers a watchlist of symbols they want to monitor. The engine activates Symbol Workers for each symbol and begins streaming indicator data:

```
POST /v1/watchlist
{
  "watchlist_name": "my_100_stock_scanner",
  "symbols": [
    "AAPL", "MSFT", "TSLA", "NVDA", "AMZN", "GOOGL", "META", "NFLX",
    "AMD", "INTC", "CRM", "ORCL", "ADBE", "NOW", "SNOW", "PLTR",
    "SPY", "QQQ", "IWM", "XLE", "XLK", "XLF", "XLV", "GLD",
    // ... up to 500 symbols on Enterprise tier
  ],
  "bar_config": { "bar_type": "TIME", "bar_size": "5m" },
  "indicators": ["delta", "imbalances", "pulse", "vwap", "volume_profile"],
  "session_filter": "RTH_ONLY"
}
Response: { watchlist_id: 'wl_abc123', symbols_activated: 100, estimated_ready_ms: 2400 }
```

i Symbol Workers spin up asynchronously. *estimated_ready_ms* is the time until all 100 workers have loaded their historical intraday state and are ready to stream live data. For 100 symbols with 5-minute bars, this is typically 2–5 seconds.

3.2 Multi-Symbol WebSocket — Single Connection, All Symbols

A developer does NOT need 100 separate WebSocket connections — one connection delivers all symbols. The topic syntax handles symbol multiplexing:

Subscribe by Watchlist (Most Efficient)

```
SUBSCRIBE watchlist:wl_abc123:pulse // Pulse signals from ALL 100 symbols
SUBSCRIBE watchlist:wl_abc123:imbalances // All imbalance events from all 100
symbols
SUBSCRIBE watchlist:wl_abc123:delta:bar // Bar delta from all 100 symbols on
bar close
SUBSCRIBE watchlist:wl_abc123:turns // Turns signals from all 100 symbols
SUBSCRIBE watchlist:wl_abc123:vwap // VWAP snapshots from all 100 symbols
every 5s
```

Subscribe to Specific Symbols Within Watchlist

```
SUBSCRIBE watchlist:wl_abc123:pulse:AAPL,TSLA,NVDA // Only these 3 from the
watchlist
```

```
SUBSCRIBE watchlist:wl_abc123:imbalances:AAPL // Only AAPL imbalances
```

Subscribe to ALL Indicators for ONE Symbol

```
SUBSCRIBE symbol:AAPL:all // All signals for AAPL – full firehose
```

Cross-Symbol Sector Stream

```
SUBSCRIBE sector:XLK:pulse // Pulse signals from ALL XLK constituents
SUBSCRIBE sector:XLK:delta:aggregate // Aggregated sector delta (sum of all XLK deltas)
```

⚠ *Subscribing to watchlist:wl_abc123:delta:live (100ms live delta) for 100 symbols generates up to 1,000 messages/second on one connection. Standard tier allows this. Use bar-close delta (watchlist:wl_abc123:delta:bar) if only end-of-bar values are needed — reduces to ~12 messages/minute per symbol.*

3.3 Multi-Symbol Signal Payload — Symbol Field

Every signal emitted on a watchlist or sector stream includes a symbol field so the consumer knows which stock triggered the signal:

```
{
  "topic": "watchlist:wl_abc123:pulse",
  "symbol": "NVDA",
  "signal_type": "MOMENTUM_JUMP",
  "direction": "LONG",
  "composite_score": 84,
  "detection_ts": 1748599847291847362,
  "bar_close_ts": 1748599800000000000,
  "price_at_signal": 891.25,
  "suggested_entry": 891.50,
  "suggested_stop": 887.00,
  "variable_scores": { "order_flow": 88, "delta": 79, "poc": 85, "imbalance": 91,
    "volume": 82, "price_action": 80, "swing": 78 },
  "watchlist_id": "wl_abc123",
  "worker_id": "worker_042",
  "processing_latency_us": 847
}
```

i *processing_latency_us* is the time in microseconds from the triggering bar close to signal emission. Developers can monitor this field to detect engine load issues. Target: < 2,000 μ s (2ms). Alert threshold: > 10,000 μ s (10ms).

4. Independent Symbol Monitoring via Automated Scripts

4.1 Per-Symbol Script Activation

Each symbol in a watchlist can have its own OFE-Script that runs independently. Scripts for AAPL and TSLA evaluate in parallel — TSLA's script does not wait for AAPL's to complete.

```
// Attach the SAME script to ALL 100 symbols in the watchlist at once:
POST /v1/scripts/signals
{
  "signal_name": "strong_pulse_long",
  "apply_to": { "watchlist_id": "wl_abc123" }, // applies to ALL 100 symbols
  "conditions": [
    { "indicator": "pulse", "field": "composite_score", "op": ">=", "value": 78 },
    { "indicator": "pulse", "field": "direction", "op": "==", "value": "LONG" },
    { "indicator": "vwap", "field": "current_price_vs_vwap", "op": "==", "value": "ABOVE_DAILY" },
    { "indicator": "imbalances", "field": "imbalance_type", "op": "in", "value": ["STACKED_BUY"] }
  ],
  "alert": { "webhook": "https://mytrader.com/signal" }
}

// Attach DIFFERENT scripts to individual symbols:
POST /v1/scripts/signals
{
  "signal_name": "aapl_earnings_play",
  "apply_to": { "symbols": ["AAPL"] }, // only AAPL
  "conditions": [
    { "indicator": "delta_surge", "field": "surge_magnitude", "op": ">=", "value": 5.0 },
    { "indicator": "vwap", "field": "daily_vwap", "op": "within_ticks", "value": 10 }
  ]
}
```

4.2 Script Evaluation Model — Parallel, Isolated, Non-Blocking

Each OFE-Script evaluator instance is completely isolated. The evaluation model guarantees:

Property	Guarantee	Implementation
Per-symbol isolation	Script for AAPL has no access to TSLA's data. Cannot accidentally cross-contaminate.	Each script evaluator receives only the signal output of its assigned symbol.

Non-blocking evaluation	A slow or complex script for one symbol does not delay signal delivery to others.	Script evaluators run as separate lightweight processes (Rust async tasks). Engine does not wait for script completion.
Ordered evaluation	Signals for the same symbol are evaluated in detection order. No race conditions.	Each symbol's script evaluator has a single-threaded queue. In-order guarantee within one symbol.
Timeout enforcement	If a user script takes > 50ms to evaluate (bad logic), it is killed and an error is logged.	Watchdog timer per evaluator. Script owner notified via portal alert.
Resource caps	CPU and memory usage per script evaluator is capped.	cgroup limits: 100ms CPU / 30-second window; 50MB RAM max.
Error isolation	A script that crashes does not affect the engine or other scripts.	Crash restarts the evaluator. The symbol's signal stream continues uninterrupted.
State per symbol	Script can maintain state across bars (e.g. count consecutive signals). Each symbol's script has independent state.	State stored in evaluator memory, scoped to symbol. Not shared with other symbols.

4.3 Script State — Tracking Across Bars Per Symbol

OFE-Scripts can maintain per-symbol state across multiple bars — enabling streak counting, cooldowns, and multi-bar pattern detection:

```
{
  "signal_name": "three_consecutive_pulse_longs",
  "apply_to": { "watchlist_id": "wl_abc123" },
  "state": {
    "consecutive_long_count": { "type": "int", "initial": 0, "reset_on":
"session_open" }
  },
  "conditions": [
    { "indicator": "pulse", "field": "direction", "op": "==", "value": "LONG" },
    { "indicator": "pulse", "field": "composite_score", "op": ">=", "value": 70 }
  ],
  "on_match": {
    "increment": "consecutive_long_count"
  },
  "on_no_match": {
    "reset": "consecutive_long_count"
  },
  "fire_when": {
    "state.consecutive_long_count": { "op": ">=", "value": 3 }
  }
}
```

i Each symbol maintains its own `consecutive_long_count`. AAPL might be at count=2 while NVDA is at count=3. The script fires for NVDA without waiting for AAPL to also reach 3.

5. Concurrent Trade Execution — Trading Multiple Symbols Simultaneously

5.1 The Concurrent Execution Challenge

When 5 of the user's 100 monitored stocks all fire a Pulse signal within the same second, the system must be able to:

- Process and emit all 5 signals with < 2ms latency each — none delayed by another
- Evaluate the user's automated trading script against all 5 simultaneously
- Route trade orders for all 5 to the broker concurrently — not sequentially
- Apply per-symbol and portfolio-level risk limits before any order reaches the broker
- Confirm all 5 orders and update portfolio state — handling partial fills and rejections
- Continue monitoring all 100 symbols for the next signal during execution

● A sequential trade execution model would process Signal 1, wait for broker confirmation, then Signal 2, wait, etc. By the time Signal 5 reaches the broker, the market has moved. Concurrent execution is architecturally mandatory for this use case.

5.2 Trade Execution Architecture

Component	Role	Concurrency Model
Signal Fan-Out	Receives signals from the Event Bus and routes to user's script evaluators	Each signal is an async message. Fan-out is O(1) per signal regardless of script count.
Script Evaluator Pool	Evaluates OFE-Script conditions per symbol. Fires trade intent when conditions met.	One async task per active symbol. All 100 evaluators run concurrently.
Risk Engine	Validates every trade intent against pre-trade risk rules before sending to broker	Synchronous gate — blocks the trade, not the signal stream. < 1ms per check.
Trade Router	Routes validated orders to the correct broker adapter	Async non-blocking. AAPL order and TSLA order sent to broker in parallel.
Broker Adapter Pool	Maintains persistent connections to one or more broker APIs	One connection per broker. Orders for different symbols queue independently.
Order State Manager	Tracks open orders, fills, partial fills, rejections across all symbols	Lock-free hash map: symbol → order state. Updated atomically on each fill event.
Position Manager	Maintains real-time portfolio positions across all symbols	Atomic counters per symbol. Portfolio-level limits checked before each new order.

5.3 Trade Execution Flow — Concurrent Multi-Symbol

When multiple signals fire simultaneously, the system follows this concurrent flow:

Scenario: 5 Pulse signals fire within 200ms on AAPL, TSLA, NVDA, MSFT, AMD

Time (ms)	What Happens	Blocking?
T+0ms	AAPL bar closes → Pulse signal detected (score=82, LONG) → emitted to Event Bus	No
T+1ms	TSLA bar closes → Pulse signal detected (score=79, LONG) → emitted to Event Bus	No
T+1ms	AAPL signal received by script evaluator → conditions pass → trade intent created	No — parallel
T+2ms	AAPL trade intent sent to Risk Engine → portfolio checks pass → order created	Risk check: < 1ms
T+2ms	NVDA bar closes → Pulse signal detected (score=85, LONG) → emitted to Event Bus	No
T+2ms	TSLA signal received → conditions pass → trade intent → Risk Engine → order	Parallel to AAPL
T+3ms	AAPL order sent to broker adapter → broker receives order	Async — no wait
T+3ms	TSLA order sent to broker adapter (simultaneously with AAPL)	Async — truly parallel
T+3ms	MSFT bar closes → signal → evaluator → risk → order → broker (all parallel)	No
T+4ms	AMD bar closes → full pipeline → broker	No
T+50ms	AAPL fill confirmation received → position updated	Async callback
T+55ms	TSLA fill confirmation → position updated	Async callback
T+60ms–200ms	NVDA, MSFT, AMD fills received and processed	All async
T+200ms	All 5 positions open. Engine continues monitoring all 100 symbols.	No interruption

✓ All 5 orders reach the broker within 4ms of the first signal. The engine continues processing ticks and detecting signals for all 100 symbols throughout — no pause, no slowdown.

5.4 Risk Management for Concurrent Multi-Symbol Trading

Without risk controls, concurrent trading of 100 symbols could expose the user to catastrophic loss if signals fire simultaneously across all 100. The Risk Engine enforces multiple layers:

Risk Control	Level	Description	Configurable
Max concurrent positions	Portfolio	Maximum number of symbols with open positions at any time. Default: 10. If 10 are open, new signals are queued but not executed until a position closes.	Yes — per user
Max position size per symbol	Symbol	Maximum dollar or contract value for any single symbol. Prevents over-concentration.	Yes — per symbol or default
Max daily loss (drawdown limit)	Portfolio	If portfolio P&L drops below configured threshold today, ALL new trade execution is suspended. Signals still fire (for monitoring), but no orders sent.	Yes — per user
Max orders per minute	Portfolio	Rate limit on order submission. Prevents runaway script loops from flooding broker API.	Yes — default: 20/min
Correlated position limit	Sector	Limits total exposure to symbols in the same sector. e.g. max 3 tech positions open simultaneously.	Yes — per sector
Pre-trade price check	Symbol	Order is rejected if the current price has moved more than N ticks from the signal's suggested_entry. Prevents chasing after latency.	Yes — staleness_ticks
Duplicate order prevention	Symbol	If an order for the same symbol and direction is already open or pending, the new signal is suppressed. No pyramid unless explicitly configured.	Yes — allow_pyramid flag
Signal quality gate	Symbol	If processing_latency_us of the triggering signal exceeds 10,000µs, the trade is skipped. Stale signals not acted upon.	Yes — max_signal_age_us

5.5 Trade Execution API — Multi-Symbol Automated Script

An OFE-Script that includes trade execution instructions triggers orders directly through the engine:

```
{
  "signal_name": "concurrent_pulse_trader",
  "apply_to": { "watchlist_id": "wl_abc123" },
  "conditions": [
    { "indicator": "pulse", "field": "composite_score", "op": ">=", "value": 75 },
    { "indicator": "pulse", "field": "direction", "op": "==", "value": "LONG" },
    { "indicator": "vwap", "field": "current_price_vs_vwap", "op": "==", "value": "ABOVE_DAILY" }
  ],
  "execute": {
```

```

    "action": "BUY",
    "quantity_type": "fixed_dollar",
    "quantity_value": 5000,           // $5,000 per symbol
    "order_type": "LIMIT",
    "limit_offset_ticks": 1,         // 1 tick above current ask
    "stop_loss_source": "suggested_stop", // use engine's suggested_stop from
signal
    "take_profit_rr": 2.0,           // 2:1 R/R target
    "broker": "interactive_brokers",
    "account_id": "U12345678"
  },
  "risk": {
    "max_concurrent_from_this_script": 5, // max 5 symbols open at once from
this script
    "staleness_ticks": 3,             // skip if price moved > 3 ticks since
signal
    "max_daily_loss_usd": 2000        // script suspends if daily loss >
$2,000
  }
}

```

5.6 Supported Broker Adapters for Automated Execution

Broker	API Protocol	Connection Type	Supported Order Types	Latency
Interactive Brokers (IB)	TWS API / IBGateway	Persistent TCP socket	Market, Limit, Stop, Stop-Limit, MOC, LOO, Bracket	< 5ms
TD Ameritrade / Schwab	Schwab API (OAuth REST)	REST + WebSocket	Market, Limit, Stop, Stop-Limit, Bracket	< 50ms
E*Trade	E*Trade API (OAuth REST)	REST	Market, Limit, Stop, Stop-Limit	< 100ms
Tradovate (Futures)	Tradovate WebSocket API	Persistent WebSocket	Market, Limit, Stop, MIT, MOO, MOC	< 10ms
Rithmic (Futures)	R Protocol API (C++)	Persistent TCP	All order types; full DOM access	< 2ms
Alpaca (Stocks)	Alpaca REST + WebSocket	REST + WebSocket	Market, Limit, Stop, Stop-Limit, Bracket, OTO	< 30ms
TradeStation	TradeStation API (REST)	REST	Market, Limit, Stop, Bracket	< 50ms
NinjaTrader Brokerage	NinjaTrader ATM API (C#)	In-process (NinjaTrader 8)	All NinjaTrader order types including ATM strategies	< 1ms

6. Cross-Symbol & Sector Analysis

6.1 Sector-Level Order Flow Aggregation

Beyond monitoring individual symbols, the engine aggregates order flow metrics across sectors and indices to surface macro-level signals that individual symbol analysis cannot detect:

Aggregate Metric	Calculation	API Endpoint / WS Topic	Trading Use
Sector Delta	Sum of bar_delta across all symbols in the sector (e.g. XLK constituents)	SUBSCRIBE sector:XLK:delta:aggregate	Rising sector delta while individual stocks are mixed = institutional rotation into sector
Sector CVD	Sum of cumulative_delta across sector	SUBSCRIBE sector:XLK:cvd:aggregate	Sector CVD divergence from sector price = institutional vs retail divergence
Breadth Indicator	% of sector symbols with positive bar_delta this bar	GET /v1/sectors/XLK/breadth? bar_size=5m	< 30% positive = broad selling; > 70% = broad buying — high conviction directional signal
Pulse Heat Map	Count and direction of Pulse signals across watchlist this bar	GET /v1/watchlist/wl_abc123/heat map	See at a glance: how many of 100 symbols are showing bullish vs bearish Pulse signals
Stacked Imbalance Density	Count of active stacked imbalance zones across watchlist	GET /v1/watchlist/wl_abc123/zones/active	High density of buy zones below current price = strong institutional support floor
Correlated Delta Surge	Detects when N or more symbols in watchlist all show delta surge simultaneously	SUBSCRIBE watchlist:wl_abc123:correlated_surge	Systemic buy or sell program — institutional algo firing across multiple names at once

6.2 Market-Wide Context Feed

The engine maintains a market-wide context feed that supplements individual symbol signals with macro conditions. Each signal payload optionally includes market_context:

```
{
  "symbol": "NVDA",
  "signal_type": "MOMENTUM_JUMP",
  "composite_score": 84,
  "market_context": {
    "spy_delta_5m": "+2847",           // S&P 500 current bar delta
    "spy_above_vwap": true,           // SPY above daily VWAP
    "vix_level": "18.4",              // current VIX level
    "sector_xlk_breadth_pct": 73,     // 73% of XLK stocks showing positive
  }
  delta
  "market_regime": "BULLISH_TREND",  // engine-classified market regime
  "time_to_macro_event_min": null    // no scheduled macro event in next 30
  min
}
```

i *market_context fields are configurable. A developer can include or exclude them per script to keep payloads lean. For strategies that only trade when SPY is above VWAP, adding spy_above_vwap as a condition to the OFE-Script filters out counter-trend signals automatically.*

7. Multi-Symbol Capacity Limits by Subscription Tier

Capability	Basic	Standard	Professional	Enterprise
Maximum symbols in watchlist	5	25	100	500+
Maximum simultaneous watchlists	1	2	5	Unlimited
Maximum active OFE-Scripts across all symbols	0	25 (1 per symbol)	500 (5 per symbol)	Unlimited
Automated trade execution	No	No	Yes — max 10 concurrent positions	Yes — configurable
Concurrent positions from automation	—	—	10	100+
Symbol Worker guaranteed CPU priority	Shared best-effort	Shared best-effort	Reserved slice (1/4 core per symbol)	Dedicated cores
Tick processing latency guarantee	< 100ms (no SLA)	< 20ms (no SLA)	< 5ms (soft SLA)	< 2ms (hard SLA)
Signal delivery latency guarantee	< 1s (no SLA)	< 200ms (no SLA)	< 20ms (soft SLA)	< 5ms (hard SLA)
Cross-symbol sector aggregation	No	No	Yes	Yes + custom groupings
Market-wide context feed	No	No	Yes	Yes + extended fields
Backtesting across all watchlist symbols	No	3 runs total/day	20 runs/day per symbol	Unlimited
Broker adapter integration	No	No	1 broker	Multiple brokers
Risk Engine access	No	No	Standard rules	Custom rules + API override
Historical tick replay for watchlist	30 days	1 year	5 years	Full history

⚠ *Symbol activation is not instant. Activating 100 new symbols requires spinning up 100 Symbol Workers, loading their intraday state, and connecting to 100 tick streams. This takes 5–15 seconds. Plan watchlist activation before market open, not during.*

8. Developer Implementation Guide — Building a 100-Symbol Scanner

8.1 Recommended Architecture for an External 100-Symbol Scanner

This is the recommended pattern for a developer who wants to monitor 100 stocks, detect signals, and trade automatically:

Step 1 — Pre-Market Setup (30 min before open)

```
# 1. Register your watchlist
POST /v1/watchlist { symbols: [...100 symbols...], bar_config: { bar_size: '5m' } }
# Response: watchlist_id = 'wl_abc123'

# 2. Attach your trading script to all 100 symbols at once
POST /v1/scripts/signals { apply_to: { watchlist_id: 'wl_abc123' }, ... }

# 3. Connect WebSocket BEFORE market open
SUBSCRIBE watchlist:wl_abc123:pulse
SUBSCRIBE watchlist:wl_abc123:imbalances
SUBSCRIBE watchlist:wl_abc123:delta:bar
# Connection ready. Buffer fills as workers initialise.
```

Step 2 — Market Open: Receive and Act on Signals

```
# Python example – one connection handles all 100 symbols
from ofe_sdk import OFEClient, WatchlistStream

client = OFEClient(api_key='ofe_pk_live_xxx', secret='ofe_sk_live_xxx')
stream = WatchlistStream(client, watchlist_id='wl_abc123')

@stream.on('pulse')
def on_pulse(msg):
    symbol = msg.symbol
    if msg.composite_score >= 75 and msg.direction == 'LONG':
        if msg.market_context['spy_above_vwap']:
            client.execute_trade(symbol=symbol, action='BUY',
                                quantity_usd=5000, stop=msg.suggested_stop)

@stream.on('turns')
def on_turns(msg):
    if msg.turn_direction == 'BEARISH_TURN' and msg.turn_strength >= 4:
        client.execute_trade(symbol=msg.symbol, action='SELL_SHORT',
                             quantity_usd=3000)

stream.run() # event loop – handles all 100 symbols concurrently
```

Step 3 — Monitoring Portfolio During Session

```

GET /v1/portfolio/positions // all open positions across all symbols
GET /v1/portfolio/pnl      // real-time P&L by symbol and total
GET /v1/watchlist/wl_abc123/heatmap?bar_size=5m // signal activity map
SUBSCRIBE portfolio:fills  // real-time fill notifications

```

8.2 Latency Budget — End-to-End for Automated Trading

From the last tick of a 5-minute bar to a confirmed order at the broker — here is the realistic latency budget for a Professional tier user:

Stage	Latency	Notes
Bar close → Signal detection	< 500 μ s	Symbol Worker evaluates all signal detectors
Signal serialization	< 100 μ s	Protobuf encoding + Redis Stream write
Event Bus → API Server	< 1 ms	Redis Stream consumer read
API Server → WebSocket delivery	< 2 ms	WebSocket frame push to client
Network (internet): Server → Client	1–50 ms	Depends on client location. Co-location: < 1ms. Retail internet: 10–50ms.
Client script evaluation	< 5 ms	OFE-Script condition evaluation in client SDK
Client → Trade Router (REST or WS)	1–50 ms	Same network dependency
Risk Engine validation	< 1 ms	Pre-trade checks
Trade Router → Broker API	1–100 ms	Depends on broker. IB: ~5ms. REST brokers: ~50ms.
Broker → Exchange	1–5 ms	Broker's internal routing
Total (best case — co-located)	~10 ms	Signal to exchange order — excellent for swing/momentum trading
Total (retail internet + IB)	~100 ms	Signal to exchange order — good for 5-minute bar strategies
Total (retail internet + REST broker)	~200 ms	Acceptable for 5-minute and above strategies

i For 5-minute bar strategies, 200ms end-to-end latency is excellent — price typically moves 1–5 ticks in 200ms on a liquid stock. For sub-minute strategies (30s bars), co-location is recommended to achieve < 20ms total latency.

9. Implementation Requirements — Engine Changes for Multi-Symbol Support

The following engine changes are required beyond the single-symbol architecture described in the main specification. These are the additions needed to support 100+ symbol concurrent monitoring:

Component	Change Required	Priority	Estimated Effort
Tick Router	NEW: Central router that receives ticks from all data provider feeds and routes to correct Symbol Worker ring buffer by instrument ID. Lock-free MPMC design.	P0 — blocking	2 weeks
Symbol Worker Pool	NEW: Thread pool manager that creates/destroys Symbol Workers on watchlist activation/deactivation. Monitors worker health and restarts on crash.	P0 — blocking	3 weeks
Symbol Worker	MODIFY: Extract all single-symbol processing into a self-contained class with its own state. Remove all global/shared state.	P0 — blocking	2 weeks
Event Bus Integration	NEW: Redis Streams or Kafka integration for signal fan-out from workers to API servers. Replaces direct function calls.	P0 — blocking	1 week
Watchlist API	NEW: REST endpoints for creating/managing watchlists. Symbol activation/deactivation. Tier limit enforcement.	P1	1 week
Multi-symbol WebSocket topics	MODIFY: Extend WebSocket topic grammar to support watchlist: and sector: prefixes. Fan-out filter by symbol.	P1	1 week
OFE-Script: apply_to watchlist	MODIFY: Script engine to accept watchlist_id or symbol array. Spawn evaluator per symbol in watchlist.	P1	1 week
OFE-Script: state per symbol	MODIFY: Script state is now scoped per symbol. State initialisation when new symbol joins watchlist.	P1	1 week
Risk Engine	NEW: Pre-trade risk validation: concurrent position limits, daily loss limits, staleness checks, correlation limits.	P1	2 weeks
Trade Router	NEW: Async order routing to broker adapters. Order state tracking. Fill confirmation handling.	P1	3 weeks
Broker Adapters	NEW: At minimum IB TWS API adapter for v1.0. Additional adapters for Tradovate, Alpaca, Rithmic per roadmap.	P1	2 weeks each

Position Manager	NEW: Real-time position tracking across all symbols. P&L calculation. Portfolio-level state.	P1	2 weeks
Sector Aggregation	NEW: Aggregate delta, CVD, breadth across sector constituents. Maintain sector indices.	P2 — post launch	2 weeks
Market Context Feed	NEW: Enrich signals with SPY delta, VIX, sector breadth, macro event proximity.	P2 — post launch	1 week
Symbol Worker Scaling	NEW: Horizontal sharding — distribute Symbol Workers across multiple servers using consistent hashing.	P3 — after 200+ symbols	4 weeks

10. Capacity Planning — Recommended Server Configuration

Deployment Phase	Symbols	Server Spec	Monthly Cost (est.)	Notes
v1.0 Launch (NinjaTrader)	Up to 50	AWS c5.2xlarge: 8 vCPU, 16GB RAM	~\$250/mo	Single server. No redundancy. 100% sufficient for NinjaTrader futures focus.
v1.5 (stocks added)	Up to 100	AWS c5.4xlarge: 16 vCPU, 32GB RAM	~\$500/mo	Single server with auto-restart. Good for 100 stock watchlists.
v2.0 (multi-platform)	Up to 250	2 × c5.4xlarge + Redis cluster	~\$1,200/mo	Active-passive failover. Redis for state + event bus.
v2.5 (Enterprise launch)	Up to 500	4 × c5.9xlarge: 36 vCPU each + Kafka + TimescaleDB	~\$4,000/mo	Active-active. Kafka for event bus. Full HA.
v3.0 (scaled)	500–2,000	8+ × c5.9xlarge + managed Kafka + RDS TimescaleDB	~\$10,000/mo	Autoscaling. Kubernetes. Zero-downtime deploys.

i These cost estimates are for AWS compute only. Data provider costs (live tick feeds for 100+ stocks) add \$500–\$5,000/month depending on provider and exchange access. Polygon.io, Refinitiv, or Interactive Brokers live data are common choices.

END OF DOCUMENT — Multi-Symbol Parallel Monitoring & Concurrent Trade Execution Spec v1.0

Sections: Scale challenge · Symbol-Sharded architecture · Parallel tick pipeline · Multi-symbol API · Independent scripts · Concurrent trade execution · Sector analysis · Capacity planning